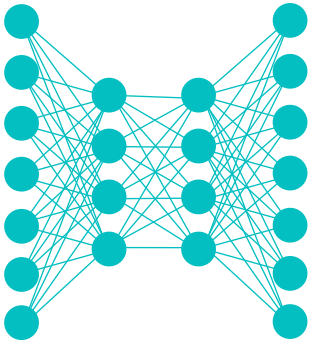
Lecture Notes for

Deep Learning

Auto-regressive

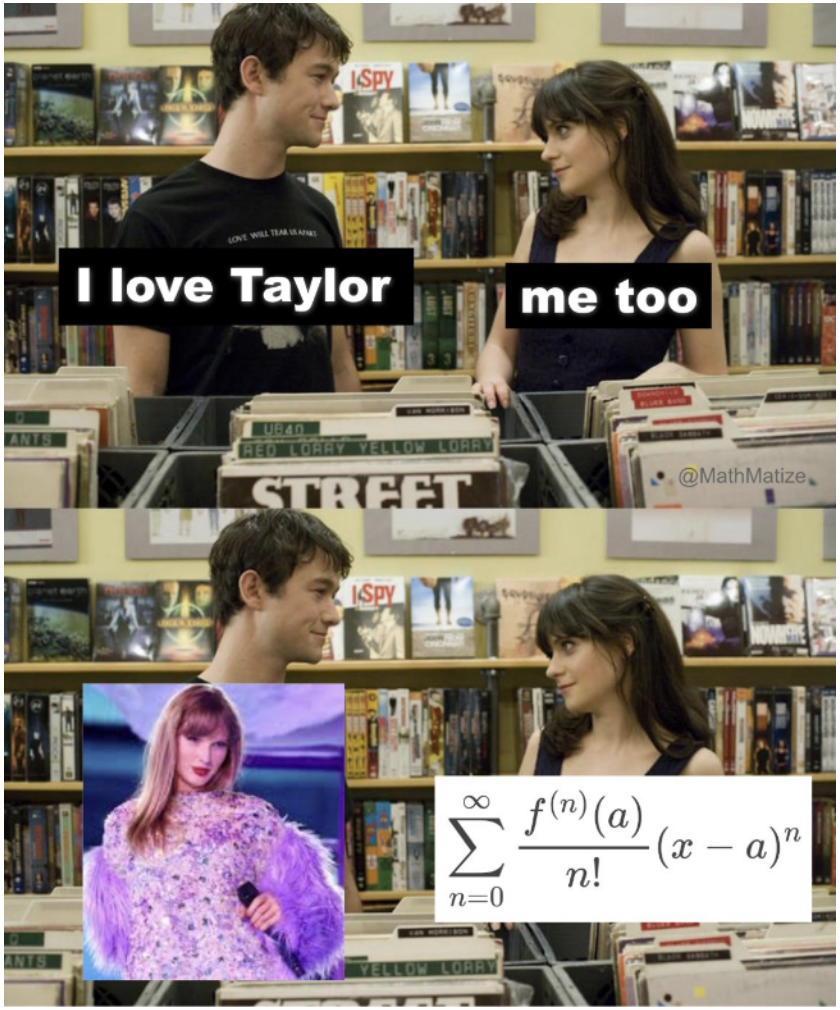
Transformers



Logistics and Agenda

- Logistics
  - Grading update
- Agenda
  - Finish BERT example
  - Paper Presentation
  - Auto-regressive Transformers
  - Town Hall (if time)

Last Time:



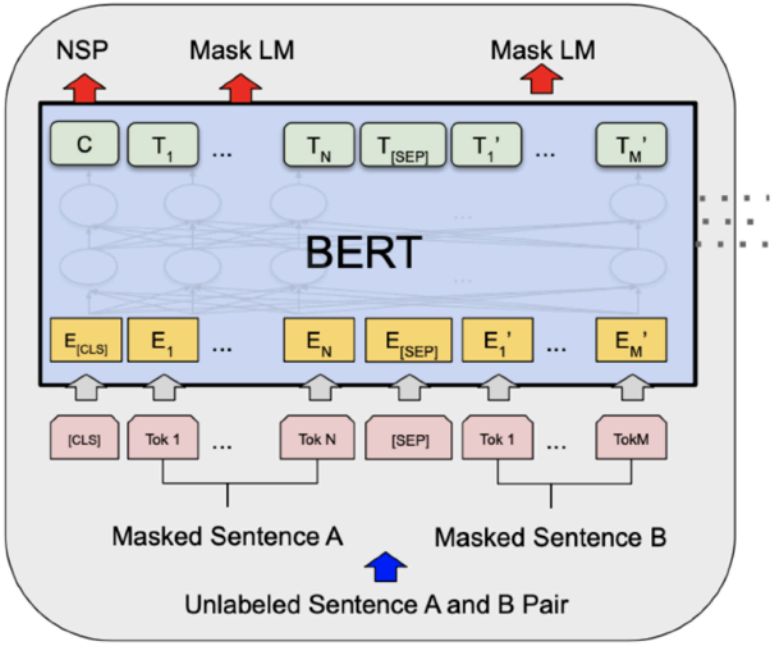
- Now we can finally understand RoPE:

$$R_a = \begin{bmatrix} \cos(a\cdot\theta_1) & -\sin(a\cdot\theta_1) & 0 & 0 & \dots & 0 & 0 \\ \sin(a\cdot\theta_1) & \cos(a\cdot\theta_1) & 0 & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \cos(a\cdot\theta_2) & -\sin(a\cdot\theta_2) & \dots & \vdots & \vdots \\ \vdots & \vdots & \sin(a\cdot\theta_2) & \cos(a\cdot\theta_2) & \dots & \vdots & \vdots \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & 0 & \dots & 0 & 0 & \cos(a\cdot\theta_{D/2}) & -\sin(a\cdot\theta_{D/2}) \\ 0 & 0 & \dots & 0 & 0 & \sin(a\cdot\theta_{D/2}) & \cos(a\cdot\theta_{D/2}) \end{bmatrix}$$

Rotate by $a\cdot\theta_1$ *(large rotate)*

Rotate by $a\cdot\theta_2$

Rotate by $a\cdot\theta_{D/2}$ *(small rotate)*



Pre-training

Fine Tuning BERT and TinyBERT

eclarson Eric Larson

20 News Groups
and IMdB

Another Great Resource, with BERT Descriptions:
https://huggingface.co/google-bert/bert-base-uncased

Main Repository:
02 BERT Transfer.ipynb

Since we are using hugging face for this, its better to use PyTorch … **Next Slide: PyTorch Basics**

Encoded inputs:
{‘input_ids’: tokens to embed,
‘token_type_ids’: segment embed,
‘attention_mask’: for causal, non-causal, truncated}

PyTorch Basics

- Setup Model
- Setup Optimizer
- Define Data Loader
- Training
  - Zero model grads
  - Feedforward batch
  - Get gradients
  - Step optimizer

Setup

```
from torch.utils.data import DataLoader
import torch
from torch.optim import Adam, AdamW

device = torch.device("mps") # or cpu or gpu
model = SetupModelArchitecture() # my custom function
model = model.to(device)
```

Utils

```
train_dataloader = DataLoader(train_data, sampler=train_sampler,…)
optimizer = AdamW( model.parameters(), lr = 2e-5)
```

Training Epoch

```
def train_epoch(model, device, train_loader, optimizer):
    model.train()

    for features, target in train_dataloader:
        features, target = features.to(device), target.to(device)
        model.zero_grad()
        output = model(features)
        # back propagation
        loss = PredefinedLoss(output, target)
        loss.backward() # perform gradient calculations
        optimizer.step() # use gradients in optimizer
```

Training Multiple Epochs

```
for i in range(epochs):
    train_epoch(model, device, train_loader, optimizer)
```

Eval with Torch and Numpy

```
model.eval()  # evaluation mode
total_loss = 0.0
all_preds = []
```

- Evaluation
  - No grad feedforward
  - Numpy, if desired

```
with torch.no_grad():
    for features, labels in eval_dataloader:
            …
        output = model(features, labels=labels) # if model has loss
        total_loss += output.loss.item() * BATCH_SIZE
        # Can save for later processing, if desired:
        all_preds.append(  output.logits.detach().cpu().numpy() )

    # after iterating through all, divide by total number of samples
    total_loss = total_loss / EVAL_SIZE
```

Paper Presentation

Causal Representation Learning on Degraded Multi-Sensor Streams

Yihao Wang, Zhongdi Wu, Ying Zhang, Eric C. Larson

17 Sep 2025 (modified: 11 Feb 2026) Submitted to ICLR 2026 Everyone Revisions BibTeX CC BY 4.0

Keywords: Causal, Multimodal, Robust Time Series Estimation, Sensor

Abstract:
Many systems require real-time fusion of multi-sensor streams to produce causal estimates that drive online decisions. These systems must distill information across sensors while contending with missing and degraded measurements. As the number of sensors grows, both observable dropouts and latent degradation become more likely, making multi-sensor, multi-task processing brittle for conventional sequential models. We propose two plug-in modules that attach to any unidirectional backbone (e.g., LSTM or causal Transformer): (i) Subchannel Hierarchical Input Embedding (SHIE) forms channel-level embeddings from fine-grained subchannels so that degraded values perturb only a local slice of the representation; (ii) Repetitive Cross-Modal Fusion Transformer (RCFT) performs iterative sensor-wise (cross-modal) attention at each time step, fusing concurrent measurements across sensors. Both modules support many-to-many estimation and are domain-agnostic with respect to loss functions and input/output shapes. We augment vanilla LSTM and Transformer backbones with SHIE and RCFT and evaluate on four multi-sensor datasets: electric grid state estimation, physical activity monitoring, room occupancy prediction, and cognitive load estimation. Across datasets, the augmented models outperform their baselines and remain accurate as missing-data rates rise far beyond those seen in training. Ablations isolate the contribution of each module, and the combined approach improves robustness without relying on separate imputation.

Supplementary Material: pdf
Primary Area: unsupervised, self-supervised, semi-supervised, and supervised representation learning
Submission Number: 9871

Rejected!

Auto-regressive Transformers

Ethics and Information Technology (2024) 26:38
https://doi.org/10.1007/s10676-024-09775-5

ORIGINAL PAPER

ChatGPT is bullshit

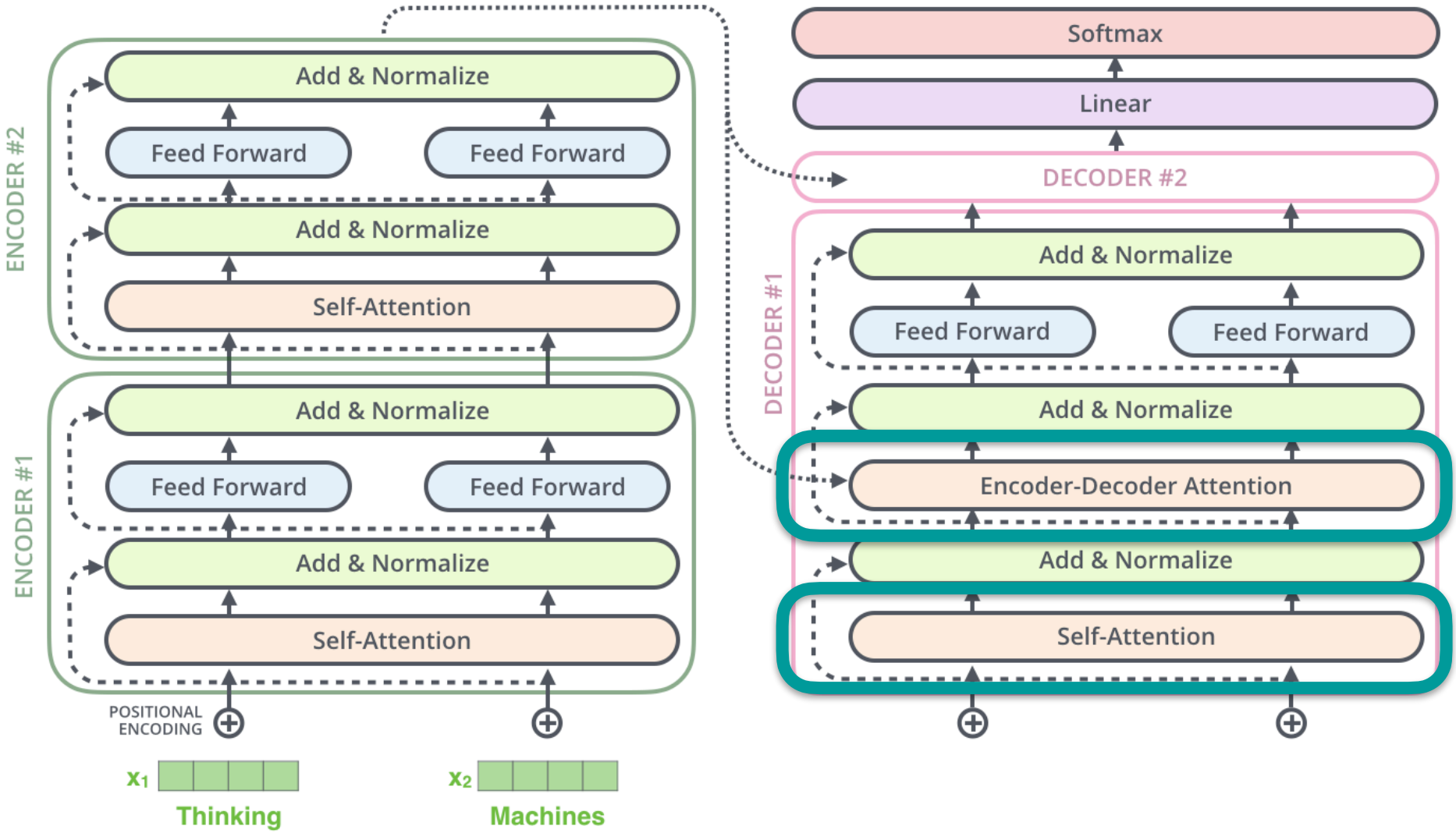
Michael Townsen Hicks[1] · James Humphries[1] · Joe Slater[1]

© The Author(s) 2024

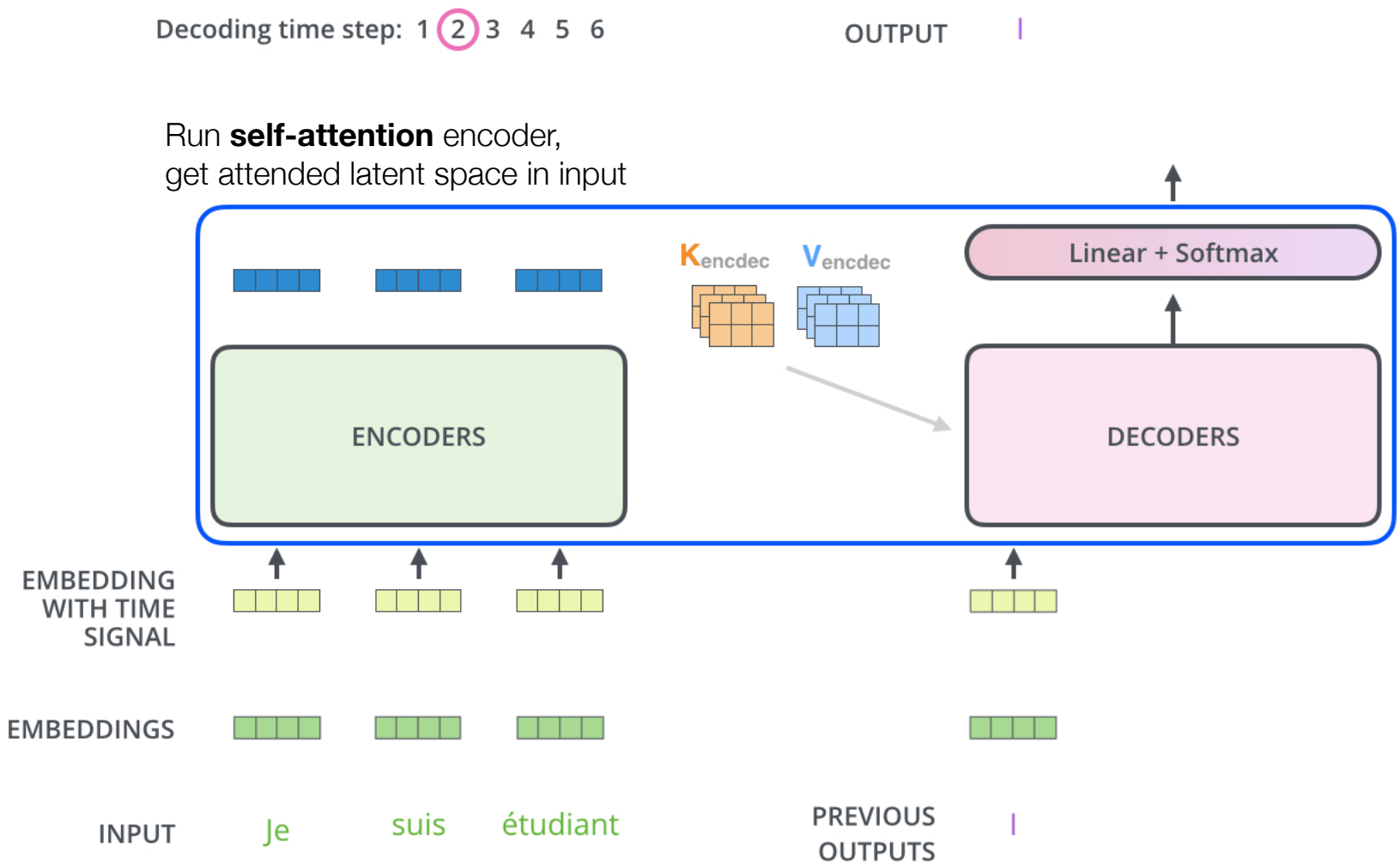
Abstract
Recently, there has been considerable interest in large language models: machine learning systems which produce human-like text and dialogue. Applications of these systems have been plagued by persistent inaccuracies in their output; these are often called "AI hallucinations". We argue that these falsehoods, and the overall activity of large language models, is better understood as *bullshit* in the sense explored by Frankfurt (On Bullshit, Princeton, 2005): the models are in an important way indifferent to the truth of their outputs. We distinguish two ways in which the models can be said to be bullshitters, and argue that they clearly meet at least one of these definitions. We further argue that describing AI misrepresentations as bullshit is both a more useful and more accurate way of predicting and discussing the behaviour of these systems.

Keywords Artificial intelligence · Large language models · LLMs · ChatGPT · Bullshit · Frankfurt · Assertion · Content

Encoders and Decoders



Encoder-Decoder Architecture



Run **self-attention** encoder,
get attended latent space in input

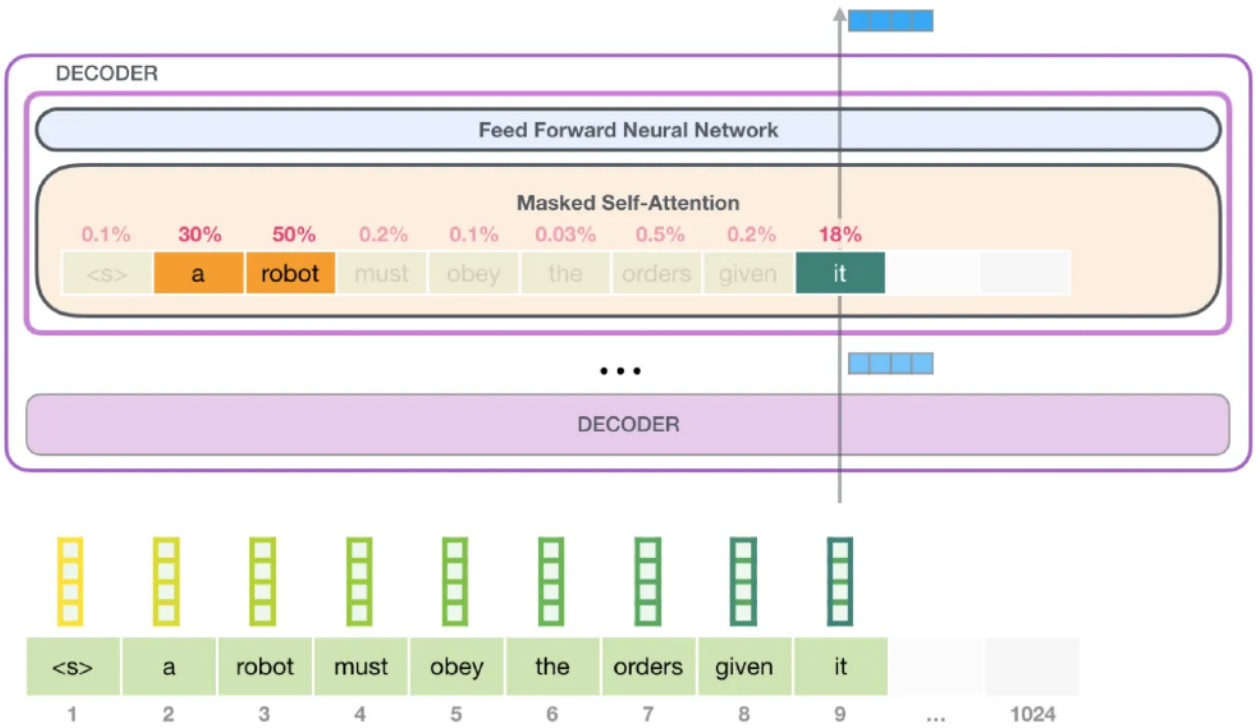
Run decoder with **cross attention** of
encoder embeddings,
run in auto regressive manner

Auto-regressive: Do We Need Encoder?

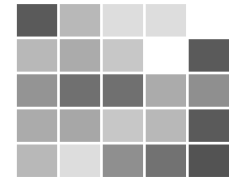
- Decoder only, text encoding happens in attention

$$\mathscr{L}_1(\mathscr{U}) = \sum_{i \in S} \log P(\underbrace{u_i}_{\text{curr}} \mid \underbrace{u_{i-k}, \ldots, u_{i-1}}_{\text{other words}}; \underbrace{\mathbf{W}}_{\text{params}})$$

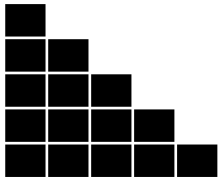
Predict the next word from unlabeled dataset



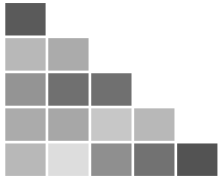
Also known as generative pre-training (GPT)



Attention Map



Mask



Masked Attention Map

Radford, et al. Improving Language Understanding by Generative Pre-Training, ArXiV 20

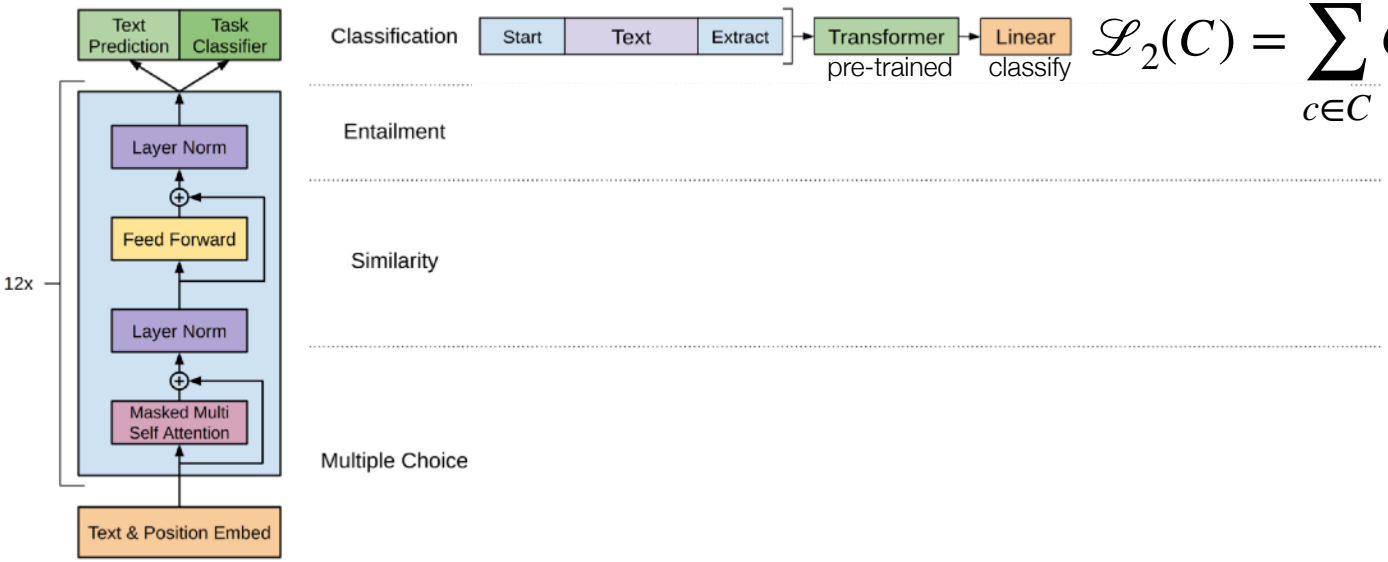
Fine Tuning with Decoder Only

- Supervised tasks after pre-training, make transformer better through various tasks, trained simultaneously

$$\sigma(h_{dec}^{ext} \cdot \mathbf{W}_c + \mathbf{b}_c)$$

new layer output for transfer learning from "extract" token

h_{dec}^{ext} is decoder embedding *only from extract token*



$$\mathscr{L}_2(C) = \sum_{c \in C} CE\left(\underbrace{y_c}_{\text{label}}, \underbrace{\sigma(h_{dec}^{ext} \cdot \mathbf{W}_c + \mathbf{b}_c)}_{\text{prediction}}\right)$$

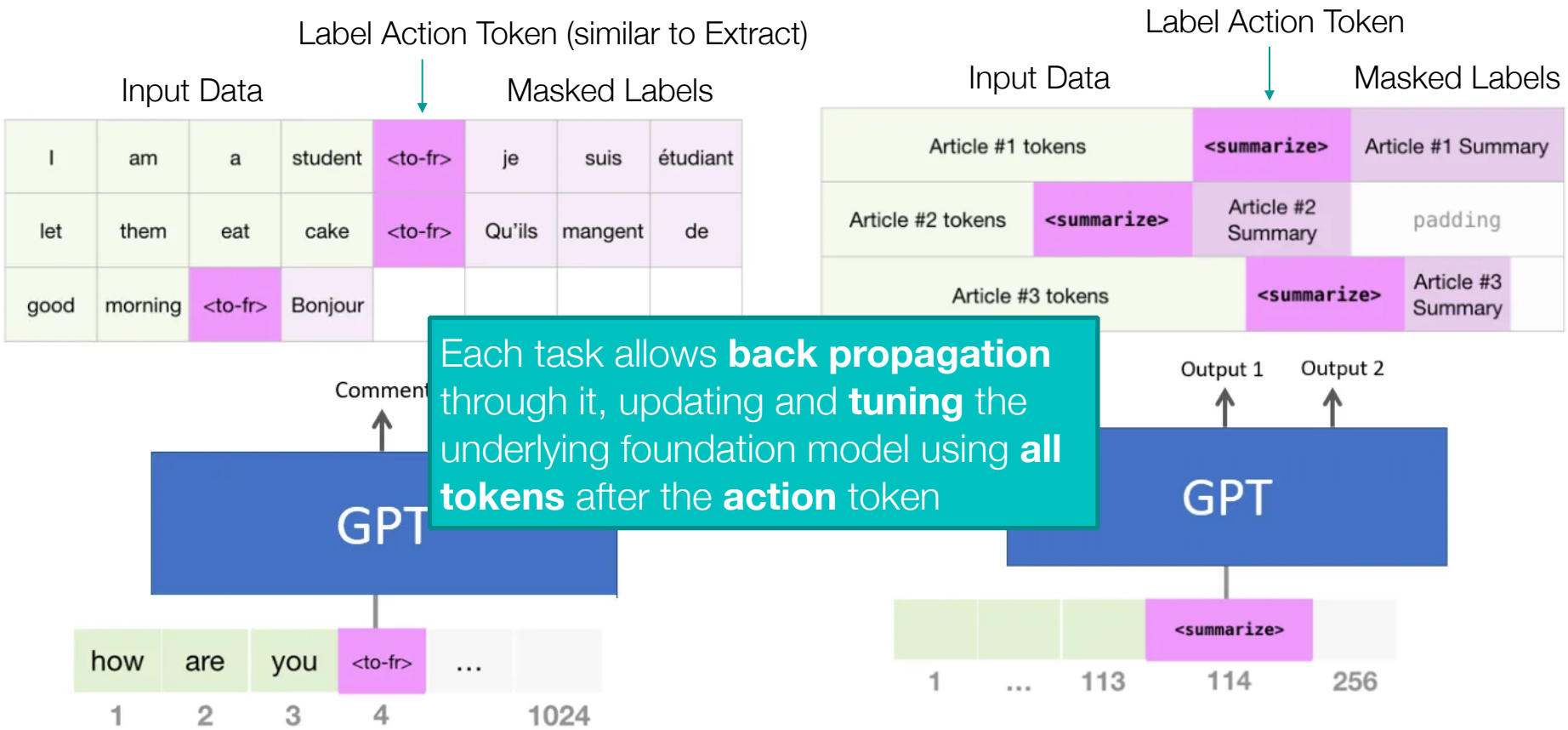
For each task, C

Each task allows **back propagation** through it, updating and **tuning** the underlying foundation model

$$\mathscr{L}_{Total} = \sum_{C \in \mathscr{C}} \underbrace{\mathscr{L}_2(C)}_{\substack{\text{task classifier} \\ \text{(Trans. \& Linear)}}} + \lambda \cdot \underbrace{\mathscr{L}_1(C)}_{\substack{\text{text predict (GPT)} \\ \text{(Trans. only, if tuning)}}}$$

Radford, et al. Improving Language Understanding by Generative Pre-Training, ArXiV 20

More than just Extract Token



Fine Tune with **"Action" Tokens** and training examples.
Train new Linear Layer to output for a specific **Extract Action**

Improving LLMs with Reinforcement Learning

- Generative pre-training allows for text generation in many different applications
- Some tasks it is not trained for, it can still generate plausible outcomes (called **emergent abilities**)
  - Many arguments exist for this being a result of natural language as input, which generalizes well to ***some*** unseen classification tasks
- To further fine tune and make answers more relevant, reinforcement learning is often used (more on this later):
  - **Game systems**: known optimal outputs, like playing chess, can be rewarded systematically
  - **Human feedback**: various tasks (like dialogue) with rewards for relevant responses and punishment for poor responses. ***Used for Chat.***
- **ChatGPT**: uses various proximal policy networks (agents) that guide next generation based on current state, needs periodic human supervision
- **DeepSeek**: mitigates human supervision in RL, opting for group sampling that balances reference distribution divergence.
  - One Insight: It helps to have many chain-of-thought examples in circulation

More Visual Sizing

Size of GPT

Unknown decoders

GPT-5

400,000 (+) Context Length
Unknown Trillion parameters (*might be smaller*)
$h_l =$ Unknown Neurons embedding (mini is about 3072 neurons)

GPT-4o
Is between 3 and 4 in size

$h_l =$ 12,288 Embedding
175B parameters

120 decoders

GPT-4

$h_l =$ 1,600
Embedding

1.5B

96 decoders

GPT-3

$h_l =$ 768
Embedding

48 decoders

GPT-2

12 decoders

117M

GPT-1

512 Context Length

1024 Context Length

2048 Context Length

8000 (+) Context Length
1.7 Trillion parameters
$h_l =$ 20,000 Neurons embedding

On par with BERT

Input Images

https://medium.com/@YanAlx/step-by-step-into-gpt-70bc4a5d8714